

MoMo SMS Transactions REST API

API Security, Endpoint Documentation, DSA Comparison and Basic Authentication Reflection

Dataset	modified_sms_v2.xml – 1,691 parsed SMS transactions
Implementation	Python <code>http.server</code> (no framework)
Authentication	HTTP Basic Authentication
Base URL	<code>http://localhost:8000</code>

Contents

- 1. Introduction to API Security
- 2. Documentation of Endpoints
- 3. Results of DSA Comparison
- 4. Reflection on Basic Authentication Limitations
- 5. Testing Summary and Conclusion

1. Introduction to API Security

An API exposes data and operations to other programs, which makes it a direct entry point into a system. API security is the set of practices that ensure only the right callers can perform the right actions on the right data, and that the service keeps working correctly when it receives malformed or hostile input. The MoMo SMS API returns financial-style transaction records, so even a small lab service should treat access control seriously.

Core principles

- **Authentication** – verifying who the caller is (here: a username and password sent with each request).
- **Authorization** – deciding what an authenticated caller may do. This API grants every valid user full CRUD access, which is acceptable for a lab but too coarse for production.
- **Transport security** – protecting data in transit with HTTPS/TLS so credentials and records cannot be read or altered on the network.
- **Input validation** – rejecting malformed JSON and missing or non-numeric fields before they reach the data store.
- **Safe error handling** – returning consistent, non-revealing errors (401, 400, 404) instead of stack traces.
- **Abuse resistance** – rate limiting, lockouts and logging to slow brute-force attempts and support auditing.

Architecture and data flow

The system has four layers, deliberately separated so each can be tested and replaced independently:

- **ETL layer** – `etl/parse_xml.py` parses `modified_sms_v2.xml` and caches 1,691 JSON records in `data/transactions.json`.
- **Storage layer** – `api/storage.py` holds an in-memory `TransactionStore` with an `id` → record dictionary for $O(1)$ lookup.
- **Handler layer** – `api/handlers_get.py` and `api/handlers_write.py` implement validation and CRUD, returning (`code`, `payload`) tuples so they can be unit-tested without a server.
- **Transport layer** – `api/app.py` runs a `ThreadingHTTPServer`, checks credentials via `api/auth.py`, and dispatches requests.

```
XML file --> parse_xml.py --> transactions.json
                                     |
                                     v
                               TransactionStore (in-memory)
                                     |
Request --> Basic Auth check --> handlers --> JSON response
                    (api/auth.py)
```

Security-relevant design decisions

- Every endpoint requires authentication; there is no anonymous access.

- Credentials are read from the environment variables `API_USER` and `API_PASS` (defaults `admin / secret123`, for lab use only) so they can change per environment without editing source.
- IDs are server-assigned under a lock, so clients cannot choose or overwrite ids on create, and concurrent requests cannot corrupt the record list.
- Missing or invalid credentials return **401** with a `WWW-Authenticate` header; invalid payloads return **400**.

2. Documentation of Endpoints

Base URL: `http://localhost:8000` **Auth:** HTTP Basic (`admin:secret123` by default). All responses use `Content-Type: application/json; charset=utf-8`.

Method	Path	Description
GET	<code>/transactions</code>	List all transactions
GET	<code>/transactions/{id}</code>	Get one transaction
POST	<code>/transactions</code>	Create a transaction
PUT	<code>/transactions/{id}</code>	Update a transaction
DELETE	<code>/transactions/{id}</code>	Delete a transaction

2.1 GET `/transactions`

Returns all transactions as a JSON array.

```
curl -u admin:secret123 http://localhost:8000/transactions
```

```
200 OK
[
  { "id": 1, "type": "RECEIVE", "amount": 2000, "sender": "Jane Smith",
    "timestamp": "2024-05-10 16:30:51", "transaction_id": "76662021700" }
]
```

```

PS D:\momo-sms-analytics> python api\app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200
-
[{"id": 1, "sms_address": "M-Money", "sms_date_ms": 1715351458724, "sms_readable_date": "10 May 2024 4:30:58 PM", "transaction_id": "76662021700", "type": "RECEIVE", "amount": 2000, "sender": "Jane Smith", "receiver": null, "timestamp": "2024-05-10 16:30:51", "body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}, {"id": 2, "sms_address": "M-Money", "sms_date_ms": 1715351506754, "sms_readable_date": "10 May 2024 4:31:46 PM", "transaction_id": "73214484437", "type": "RECEIVE", "amount": 2000, "sender": "Jane Smith", "receiver": null, "timestamp": "2024-05-10 16:31:46", "body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:31:46. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 73214484437."}]

```

Figure 1. GET /transactions with valid credentials returns 200 and the JSON list

2.2 GET /transactions/{id}

Returns a single transaction, or 404 if the id does not exist.

```
curl -i -u admin:secret123 http://localhost:8000/transactions/1
```

200 OK

```
{ "id": 1, "type": "RECEIVE", "amount": 2000 }
```

404 Not Found

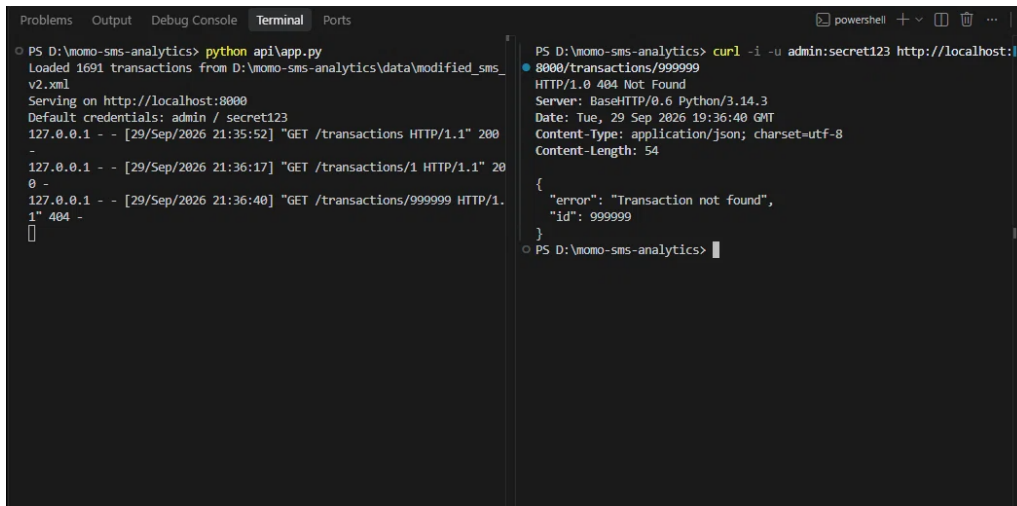
```
{ "error": "Transaction not found", "id": 999999 }
```

```

PS D:\momo-sms-analytics> python api\app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200
-
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200
-
{"id": 1, "sms_address": "M-Money", "sms_date_ms": 1715351458724, "sms_readable_date": "10 May 2024 4:30:58 PM", "transaction_id": "76662021700", "type": "RECEIVE", "amount": 2000, "sender": "Jane Smith", "receiver": null, "timestamp": "2024-05-10 16:30:51", "body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}

```

Figure 2. GET /transactions/1 returns 200 OK with the record



```
PS D:\momo-sms-analytics> python api/app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:40] "GET /transactions/999999 HTTP/1.1" 404 -
[]

PS D:\momo-sms-analytics> curl -i -u admin:secret123 http://localhost:8000/transactions/999999
HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.14.3
Date: Tue, 29 Sep 2026 19:36:40 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 54

{"error": "Transaction not found",
 "id": 999999}
PS D:\momo-sms-analytics>
```

Figure 3. GET /transactions/999999 returns 404 Not Found

2.3 POST /transactions

Creates a transaction. type and a numeric amount are required; the server assigns the id.

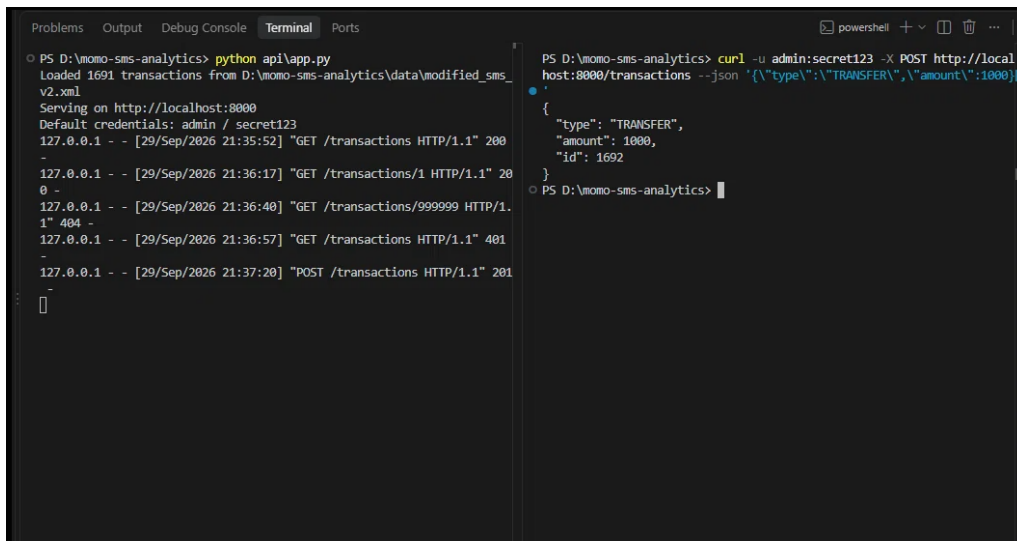
```
curl -u admin:secret123 -X POST http://localhost:8000/transactions \
  --json '{"type":"TRANSFER","amount":1000}'
```

201 Created

```
{ "type": "TRANSFER", "amount": 1000, "id": 1692 }
```

400 Bad Request

```
{ "error": "Missing required field: amount" }
```



```
PS D:\momo-sms-analytics> python api/app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:40] "GET /transactions/999999 HTTP/1.1" 404 -
127.0.0.1 - - [29/Sep/2026 21:36:57] "GET /transactions HTTP/1.1" 401 -
127.0.0.1 - - [29/Sep/2026 21:37:20] "POST /transactions HTTP/1.1" 201 -
[]

PS D:\momo-sms-analytics> curl -u admin:secret123 -X POST http://localhost:8000/transactions --json '{"type":"TRANSFER","amount":1000}'
HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.14.3
Date: Tue, 29 Sep 2026 19:36:40 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 54

{"type": "TRANSFER",
 "amount": 1000,
 "id": 1692}
PS D:\momo-sms-analytics>
```

Figure 4. POST /transactions returns 201 Created with server-assigned id 1692

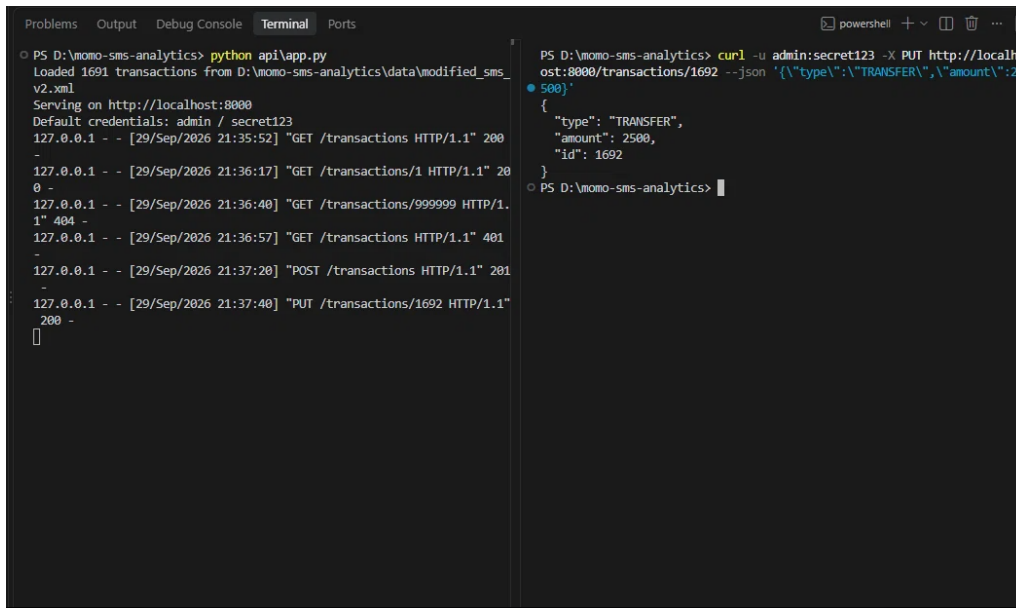
2.4 PUT /transactions/{id}

Updates an existing transaction using the same validation rules as POST.

```
curl -u admin:secret123 -X PUT http://localhost:8000/transactions/1692 \
  --json '{"type":"TRANSFER","amount":2500}'
```

200 OK

```
{ "type": "TRANSFER", "amount": 2500, "id": 1692 }
```



```
PS D:\momo-sms-analytics> python api\app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:40] "GET /transactions/999999 HTTP/1.1" 404 -
127.0.0.1 - - [29/Sep/2026 21:36:57] "GET /transactions HTTP/1.1" 401 -
127.0.0.1 - - [29/Sep/2026 21:37:20] "POST /transactions HTTP/1.1" 201 -
127.0.0.1 - - [29/Sep/2026 21:37:40] "PUT /transactions/1692 HTTP/1.1" 200 -

PS D:\momo-sms-analytics> curl -u admin:secret123 -X PUT http://localhost:8000/transactions/1692 --json '{"type":"TRANSFER","amount":2500,"id":1692}'
{"type": "TRANSFER",
 "amount": 2500,
 "id": 1692}
PS D:\momo-sms-analytics>
```

Figure 5. PUT /transactions/1692 returns 200 OK with the updated amount

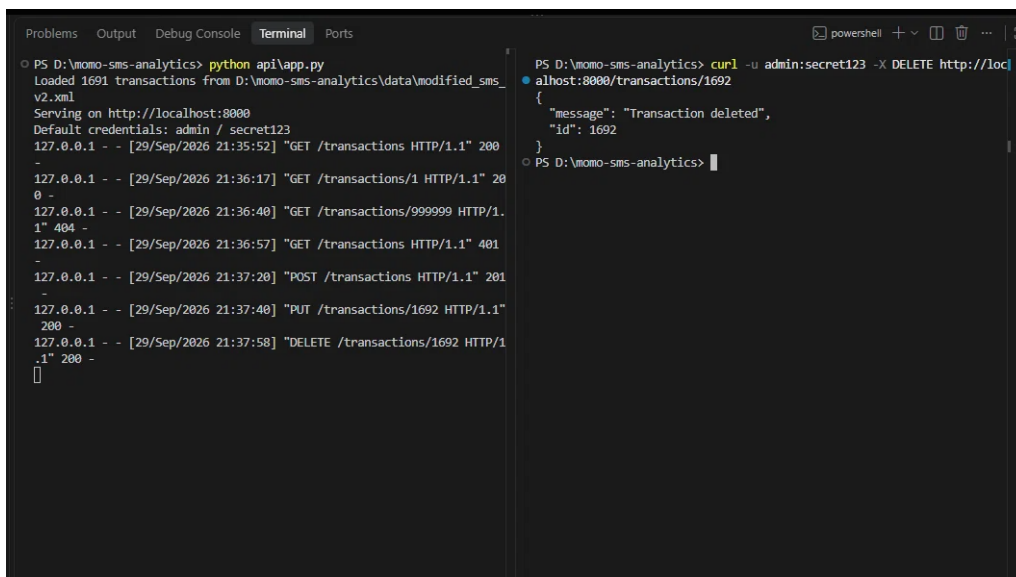
2.5 DELETE /transactions/{id}

Removes a transaction.

```
curl -u admin:secret123 -X DELETE http://localhost:8000/transactions/1692
```

200 OK

```
{ "message": "Transaction deleted", "id": 1692 }
```



```
PS D:\momo-sms-analytics> python api\app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:40] "GET /transactions/999999 HTTP/1.1" 404 -
127.0.0.1 - - [29/Sep/2026 21:36:57] "GET /transactions HTTP/1.1" 401 -
127.0.0.1 - - [29/Sep/2026 21:37:20] "POST /transactions HTTP/1.1" 201 -
127.0.0.1 - - [29/Sep/2026 21:37:40] "PUT /transactions/1692 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:37:58] "DELETE /transactions/1692 HTTP/1.1" 200 -

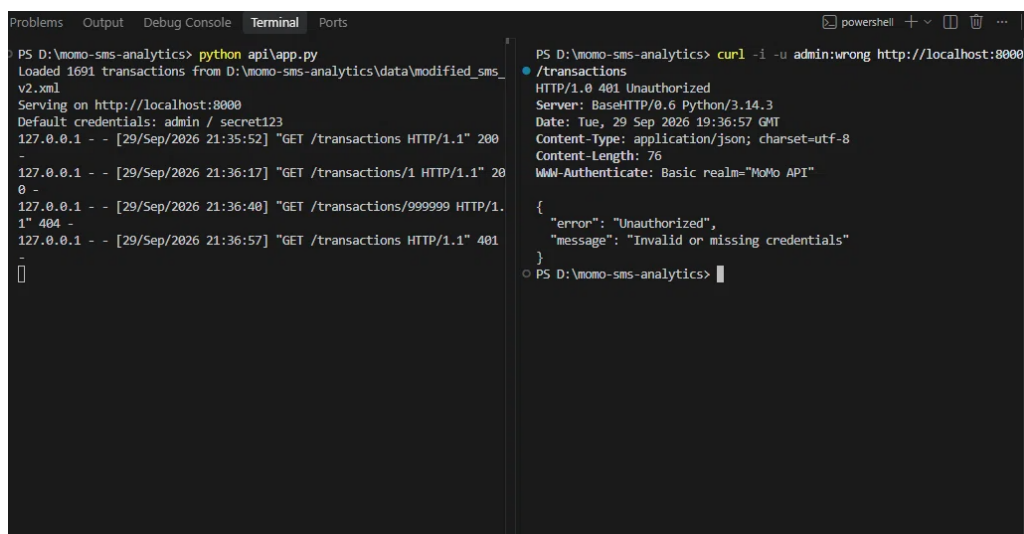
PS D:\momo-sms-analytics> curl -u admin:secret123 -X DELETE http://localhost:8000/transactions/1692
{"message": "Transaction deleted",
 "id": 1692}
PS D:\momo-sms-analytics>
```

Figure 6. DELETE /transactions/1692 returns 200 OK

2.6 Status codes and error format

Code	Meaning
200	OK
201	Created
400	Validation error or invalid JSON
401	Unauthorized (missing or wrong credentials)
404	Transaction not found

Errors always return a JSON object with an `error` key, plus extra detail (such as the `id`) when helpful.



```
PS D:\momo-sms-analytics> python api/app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
127.0.0.1 - - [29/Sep/2026 21:35:52] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:17] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 21:36:40] "GET /transactions/999999 HTTP/1.1" 404 -
127.0.0.1 - - [29/Sep/2026 21:36:57] "GET /transactions HTTP/1.1" 401 -

PS D:\momo-sms-analytics> curl -i -u admin:wrong http://localhost:8000/transactions
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.14.3
Date: Tue, 29 Sep 2026 19:36:57 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 76
WWW-Authenticate: Basic realm="MoMo API"

{"error": "Unauthorized",
 "message": "Invalid or missing credentials"
}
```

Figure 7. Wrong credentials return 401 Unauthorized with the WWW-Authenticate header

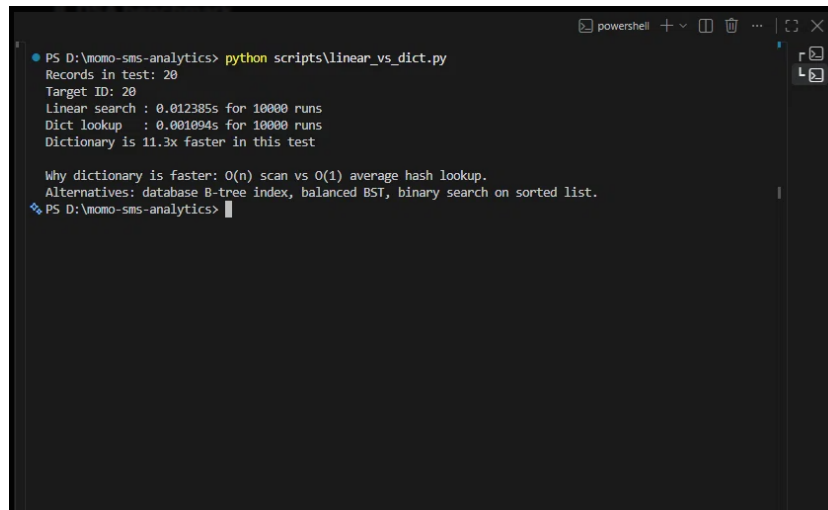
3. Results of DSA Comparison

The API needs fast repeated lookups by id. `scripts/linear_vs_dict.py` compares two strategies:

- **Linear search** – scans the record list and returns the first matching id. Time complexity $O(n)$.
- **Dictionary lookup** – reads from a dict of `id` → record. $O(1)$ on average.

Benchmark results

Metric	Linear search	Dictionary lookup
Records in test	20	20
Target id	20 (worst case: last record)	20
Total time, 10,000 runs	0.012385 s	0.001094 s
Complexity	$O(n)$	$O(1)$ average
Result	Baseline	11.3x faster



```
PS D:\momo-sms-analytics> python scripts\linear_vs_dict.py
Records in test: 20
Target ID: 20
Linear search : 0.012385s for 10000 runs
Dict lookup   : 0.001094s for 10000 runs
Dictionary is 11.3x faster in this test

Why dictionary is faster: O(n) scan vs O(1) average hash lookup.
Alternatives: database B-tree index, balanced BST, binary search on sorted list.
PS D:\momo-sms-analytics>
```

Figure 8. Linear search vs dictionary lookup, 20 records, 10,000 runs

Analysis

Linear search must examine records one by one, so its cost grows with the list. A dictionary hashes the key and jumps directly to the value, so its cost stays roughly constant. The 11.3x gap was measured on only 20 records and the target was the last one (worst case for linear search). With the real dataset of 1,691 records, or millions in production, the gap would widen: linear search degrades proportionally while dictionary lookup remains nearly flat.

This is why TransactionStore keeps a dict for id lookups and a list to preserve insertion order for GET /transactions. The trade-off is extra memory for the dictionary, which is negligible at this scale.

Alternatives if the access pattern changes

- **Balanced BST / Red-Black tree** – $O(\log n)$ and keeps sorted order, useful for range scans.
- **Database B-tree index** (e.g. PostgreSQL) – the natural choice once data leaves memory.
- **Binary search on a sorted list** – $O(\log n)$ if records are static.
- **Trie** – for prefix searches on string keys such as `transaction_id`.

4. Reflection on Basic Authentication Limitations

HTTP Basic Authentication was used because the assignment requires it. It is simple and universally supported, which makes it easy to implement and test with curl, but building the API made its weaknesses clear.

Why Basic Auth is weak

- **Base64 is encoding, not encryption.** Anyone who captures the Authorization header can decode the username and password instantly.
- **Credentials travel with every request**, multiplying the chances of exposure through logs, proxies or interception.
- **No expiry, revocation or scope.** A stolen password works until it is changed, and every client has the same full privileges.
- **Plain HTTP is exposed.** Without HTTPS, requests are open to replay and man-in-the-middle attacks.
- **No built-in rate limiting**, so brute-force password guessing must be blocked separately.
- **Weak defaults.** The default admin / secret123 pair is fine for a lab but would be trivial to guess in a real deployment.

Stronger alternatives

Approach	How it improves on Basic Auth
JWT	Signed, short-lived tokens that the server can expire; the password is not sent on every call.
OAuth2	Delegated authorization with fine-grained scopes and refresh tokens.
API keys with HMAC	Client signs each request with a shared secret and a nonce; replayed requests are rejected.
Mutual TLS	Client and server authenticate each other with certificates.

Reflection

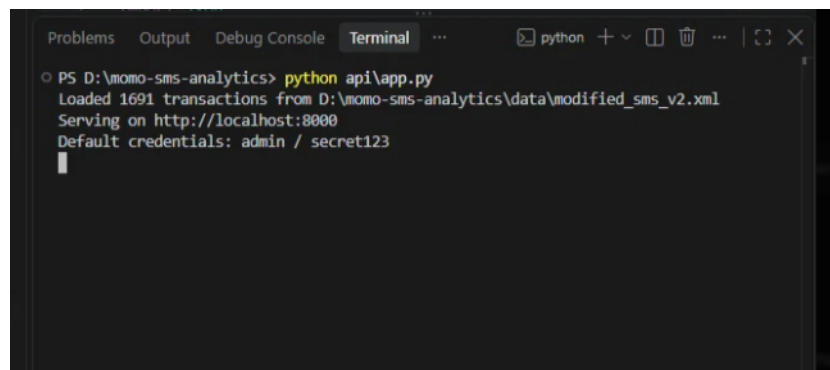
Basic Auth over plain HTTP is only acceptable in a controlled lab environment like this one. For anything real, the minimum would be to serve the API over HTTPS, replace the default credentials, add rate limiting, and move to token-based authentication (JWT or OAuth2) with per-user roles so that read access and write access can be granted separately. All of the alternatives above assume HTTPS in practice, so transport security is the first fix, not an optional extra.

5. Testing Summary and Conclusion

The server was run on `http://localhost:8000` with 1,691 transactions loaded. All five endpoints were exercised with curl against live HTTP traffic, and unit tests covered the parser, auth module, handlers and DSA benchmark.

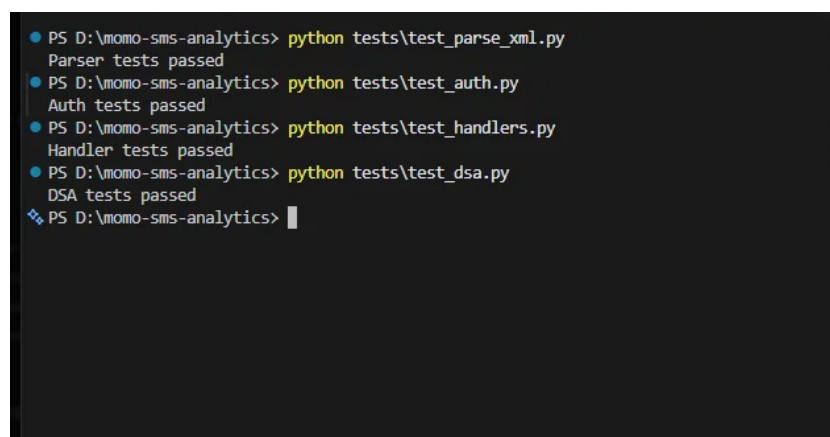
Test	Expected result
GET /transactions	200 OK, JSON array
GET /transactions/1	200 OK, single record
GET /transactions/999999	404 Not Found
Request with wrong credentials	401 with WWW-Authenticate header
POST /transactions	201 Created, id 1692
PUT /transactions/1692	200 OK, updated record
DELETE /transactions/1692	200 OK, deletion message

The server log shows the status sequence 200, 200, 404, 401, 201, 200, 200, matching the table above.



```
PS D:\momo-sms-analytics> python api\app.py
Loaded 1691 transactions from D:\momo-sms-analytics\data\modified_sms_v2.xml
Serving on http://localhost:8000
Default credentials: admin / secret123
```

Figure 9. Server startup: 1,691 transactions loaded



```
PS D:\momo-sms-analytics> python tests\test_parse_xml.py
Parser tests passed
PS D:\momo-sms-analytics> python tests\test_auth.py
Auth tests passed
PS D:\momo-sms-analytics> python tests\test_handlers.py
Handler tests passed
PS D:\momo-sms-analytics> python tests\test_dsa.py
DSA tests passed
PS D:\momo-sms-analytics>
```

Figure 10. Parser, auth, handler and DSA unit tests all pass

Constraints enforced

- Authentication is required on every endpoint.
- POST and PUT reject payloads missing type or amount, or with a non-numeric amount (400).

- IDs are server-assigned; create, update and delete are guarded by a `threading.Lock`.
- All errors share one JSON shape with an `error` key.

Conclusion

The implementation delivers a working XML to JSON parser, a REST API on plain `http` server with the five CRUD endpoints, Basic Authentication with proper 401 responses, full endpoint documentation, and a DSA comparison showing dictionary lookup is 11.3x faster than linear search on the test set. The report also documents why Basic Auth is unsuitable for production and which mechanisms should replace it.